

Experiment No.	2
Experiment Title	Email Spam / Ham Classification using Naive Bayes and K-Nearest Neighbours (KNN) ¹
Student Name	Mohamed Farih J
Register Number	3122247001035
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	August 2, 2026

1 Objective

The objective of this experiment is to classify email messages as **Spam** or **Ham (not spam)** using the Spambase dataset, by implementing and comparing three variants of the Naive Bayes classifier — Gaussian, Multinomial, and Bernoulli — against the K-Nearest Neighbours (KNN) classifier. The experiment further studies the effect of the neighbourhood size k , tunes KNN hyperparameters using GridSearchCV and RandomizedSearchCV, compares KD-Tree and Ball-Tree neighbour-search structures, validates models using 5-fold cross-validation, and analyzes the theoretical versus practical computational complexity of each algorithm.

2 Problem Statement

Input: A set of 4601 email instances, each represented by 57 numerical features derived from word frequencies, character frequencies, and capital-letter run-length statistics extracted from the email body, along with a binary class label.

Output: A predicted binary label for each email — 1 (Spam) or 0 (Ham) — produced by each of the four trained classifiers (GaussianNB, MultinomialNB, BernoulliNB, and KNN).

Objective: To build accurate, well-validated classifiers that discriminate spam from legitimate email using these frequency-based features, and to empirically compare the trained models on standard classification metrics, cross-validated generalization performance, and computational cost, in order to identify the model best suited for practical spam-filtering deployment.

3 Dataset Description

Dataset Name	Spambase Dataset
Dataset Source	Kaggle (https://www.kaggle.com/datasets/somesh24/spambase), originally from the UCI Machine Learning Repository
Number of Samples	4601 (before cleaning); 4210 after duplicate removal
Number of Features	57 numerical features (48 word-frequency, 6 character-frequency, 3 capital-run-length statistics)

Number of Classes	2 (Spam = 1, Ham = 0)
Missing Values	None (0 missing values found)
Duplicate Rows	391 duplicate rows found and removed (4601 → 4210 rows)
Train-Test Split	80% train (3368 samples) / 20% test (842 samples), stratified, <code>random_state=42</code>

4 Theory

4.1 Email Spam Classification

Email spam classification is a supervised binary text-classification problem in which each email is represented as a feature vector (commonly derived from word frequencies, character frequencies, and structural statistics) and a model is trained to predict whether the email is unsolicited/spam or legitimate/ham. It is one of the most widely deployed applications of machine learning, forming the basis of production email-filtering systems.

4.2 Supervised Learning

Supervised learning is a machine learning paradigm in which a model is trained on a labeled dataset $\{(x_i, y_i)\}_{i=1}^n$, where $x_i \in \mathbb{R}^d$ is a feature vector and y_i is the corresponding class label. The goal is to learn a mapping $f : \mathbb{R}^d \rightarrow \mathcal{Y}$ that generalizes well to unseen data. In this experiment, $\mathcal{Y} = \{0, 1\}$ (Ham, Spam), making this a binary classification task.

4.3 Bayes' Theorem

Bayes' Theorem relates the conditional and marginal probabilities of random events and forms the foundation of the Naive Bayes classifier:

$$P(y | x) = \frac{P(x | y) P(y)}{P(x)} \quad (1)$$

where $P(y)$ is the prior probability of class y , $P(x | y)$ is the likelihood of observing features x given class y , and $P(y | x)$ is the posterior probability of class y given the observed features x .

4.4 Naive Bayes Classifier

The Naive Bayes classifier extends Bayes' Theorem with the *naive* assumption that features are conditionally independent given the class label:

$$P(y | x_1, x_2, \dots, x_d) \propto P(y) \prod_{i=1}^d P(x_i | y) \quad (2)$$

The predicted class is the one that maximizes the posterior probability:

$$\hat{y} = \arg \max_y P(y) \prod_{i=1}^d P(x_i | y) \quad (3)$$

Despite the independence assumption rarely holding exactly in practice, Naive Bayes classifiers are computationally efficient, require little training data, and often perform surprisingly well, particularly on high-dimensional, frequency-based data such as Spam-base.

4.4.1 Gaussian Naive Bayes

GaussianNB assumes that, for each class y , the continuous feature x_i is normally distributed:

$$P(x_i | y) = \frac{1}{\sqrt{2\pi\sigma_y^2}} \exp\left(-\frac{(x_i - \mu_y)^2}{2\sigma_y^2}\right) \quad (4)$$

where μ_y and σ_y^2 are the mean and variance of feature x_i estimated from the training samples of class y . It is well suited to standardized, continuous features.

4.4.2 Multinomial Naive Bayes

MultinomialNB models each feature as a count or frequency drawn from a multinomial distribution, and is commonly applied to text/word-frequency data:

$$P(x_i | y) = \frac{(\sum_i x_i)!}{\prod_i x_i!} \prod_i \theta_{y,i}^{x_i} \quad (5)$$

where $\theta_{y,i}$ is the probability of feature i occurring in class y . It strictly requires non-negative feature values, which is naturally satisfied by the raw Spambase word/character frequency features.

4.4.3 Bernoulli Naive Bayes

BernoulliNB assumes each feature is a binary indicator (e.g. word present/absent), and internally binarizes continuous inputs using a threshold:

$$P(x_i | y) = x_i p_{y,i} + (1 - x_i)(1 - p_{y,i}) \quad (6)$$

where $p_{y,i}$ is the probability that feature i is present given class y . Unlike MultinomialNB, BernoulliNB explicitly penalizes the *non-occurrence* of a feature, which can be advantageous when the absence of certain words/characters is itself informative.

4.5 K-Nearest Neighbours (KNN)

KNN is a non-parametric, instance-based (“lazy”) learning algorithm. It performs no explicit training beyond storing the training data; classification of a new point x is performed by finding the k training points closest to x (under a chosen distance metric) and assigning the majority class among them:

$$\hat{y} = \text{mode}(\{y_i : x_i \in N_k(x)\}) \quad (7)$$

where $N_k(x)$ denotes the set of k nearest neighbours of x . When `weights='distance'` is used, closer neighbours are given proportionally greater influence on the vote.

4.6 Distance Metrics

4.6.1 Euclidean Distance

The Euclidean (or L_2) distance between two points x and z in d -dimensional space is:

$$d_{\text{Euclidean}}(x, z) = \sqrt{\sum_{i=1}^d (x_i - z_i)^2} \quad (8)$$

4.6.2 Manhattan Distance

The Manhattan (or L_1 , city-block) distance is:

$$d_{\text{Manhattan}}(x, z) = \sum_{i=1}^d |x_i - z_i| \quad (9)$$

Manhattan distance is often more robust to high-dimensional, sparse, or outlier-prone data than Euclidean distance, which is relevant for the 57-dimensional Spambase feature space.

4.7 KD-Tree

A KD-Tree (K-Dimensional Tree) is a binary spatial-partitioning data structure that recursively splits the feature space along axis-aligned hyperplanes, cycling through dimensions at each level of the tree. It enables nearest-neighbour queries in $O(\log n)$ average time in low dimensions by pruning large portions of the search space, though its effectiveness degrades as dimensionality increases.

4.8 Ball-Tree

A Ball-Tree partitions the feature space using nested hyperspheres (“balls”) rather than axis-aligned splits. Each node represents a ball that bounds a subset of points, and the triangle inequality is used to prune branches during search. Ball-Trees tend to be more robust than KD-Trees in higher-dimensional spaces, since spherical partitions do not depend on axis alignment.

4.9 GridSearchCV

GridSearchCV performs an exhaustive search over a specified hyperparameter grid, evaluating every possible combination of parameter values using cross-validation and selecting the combination that maximizes the chosen scoring metric. It guarantees finding the best configuration within the grid, at the cost of computation time that grows multiplicatively with the number of hyperparameters and their candidate values.

4.10 RandomizedSearchCV

RandomizedSearchCV samples a fixed number of hyperparameter combinations at random from the specified distributions/grid, evaluating each with cross-validation. It trades

an exhaustive guarantee for substantially reduced computation time, and scales far better to large hyperparameter spaces, since its cost is controlled directly by the number of iterations (`n_iter`) rather than the size of the grid.

4.11 K-Fold Cross Validation

K-Fold Cross Validation partitions the training data into K equally sized folds. The model is trained on $K - 1$ folds and validated on the remaining fold, repeated K times so that every fold serves as the validation set exactly once. The reported performance is the average across all K folds, giving a more robust estimate of generalization performance than a single train/validation split. **Stratified** K-Fold additionally preserves the class proportions in each fold, which is important for the moderately imbalanced Spambase dataset (approximately 60% ham, 40% spam).

4.12 ROC Curve

The Receiver Operating Characteristic (ROC) curve plots the True Positive Rate (Recall) against the False Positive Rate at varying classification thresholds. The Area Under the ROC Curve (ROC-AUC) summarizes this curve into a single scalar between 0 and 1, measuring the model's ability to rank a randomly chosen positive instance above a randomly chosen negative instance, independent of any single decision threshold.

4.13 Precision-Recall Curve

The Precision-Recall curve plots Precision against Recall across classification thresholds. It is particularly informative for imbalanced datasets, since it focuses on the performance of the classifier on the positive (minority) class without being influenced by the (typically large) number of true negatives.

4.14 Confusion Matrix

A confusion matrix is a 2×2 table (for binary classification) that cross-tabulates predicted versus actual class labels, reporting True Positives (TP), False Positives (FP), True Negatives (TN), and False Negatives (FN). It is the basis from which Accuracy, Precision, Recall, and F1-score are computed.

4.15 Performance Metrics

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (10)$$

$$\text{Precision} = \frac{TP}{TP + FP} \quad (11)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (12)$$

$$\text{F1-score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (13)$$

ROC-AUC is computed as the area under the ROC curve described above. Together, these metrics characterize different aspects of classifier performance: Accuracy summarizes overall correctness, Precision measures how trustworthy a “spam” prediction is, Recall measures how completely spam is being caught, F1 balances the two, and ROC-AUC summarizes ranking quality independent of threshold.

5 Exploratory Data Analysis

Exploratory Data Analysis (EDA) was performed to understand the class balance, feature distributions, correlations, and potential outliers in the Spambase dataset before modeling.

- Dataset overview: 4210 emails (after de-duplication), 57 numeric features, 1 binary target column.
- Statistical summary: computed via `X.describe()` across all 57 features (feature means and standard deviations vary substantially, motivating feature scaling).
- Missing value analysis: 0 missing values found across all columns.
- Class distribution: 60.1% Ham, 39.9% Spam — a moderate, not severe, class imbalance.
- Correlation matrix: computed among the top 15 features most correlated with the target.
- Feature distribution: examined via histograms and boxplots of the most target-correlated features.

Class Distribution

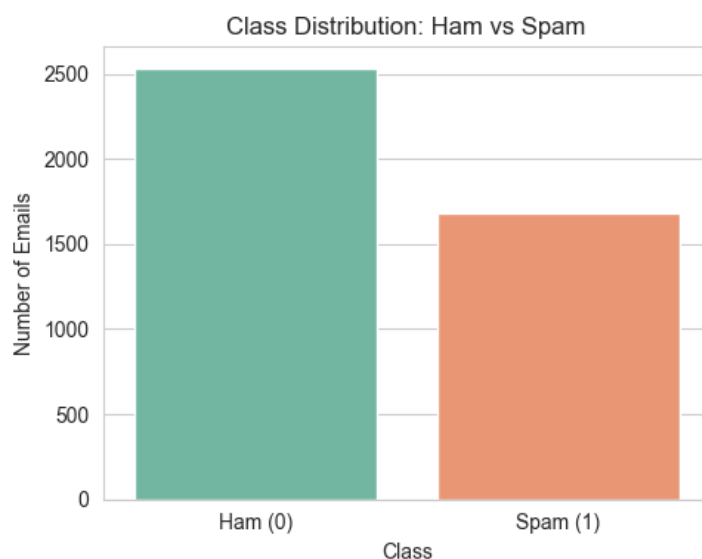


Figure 1: Class distribution of Ham (0) vs Spam (1) emails in the cleaned Spambase dataset.

Inference: The dataset contains 60.1% Ham and 39.9% Spam emails, indicating a moderate class imbalance rather than a severe one. This justifies the use of stratified train-test splitting and stratified cross-validation folds to preserve class proportions, and motivates reporting Precision, Recall, and F1-score alongside Accuracy, since Accuracy alone can be misleading under class imbalance. The imbalance is not severe enough to require resampling techniques such as SMOTE.

Correlation Heatmap

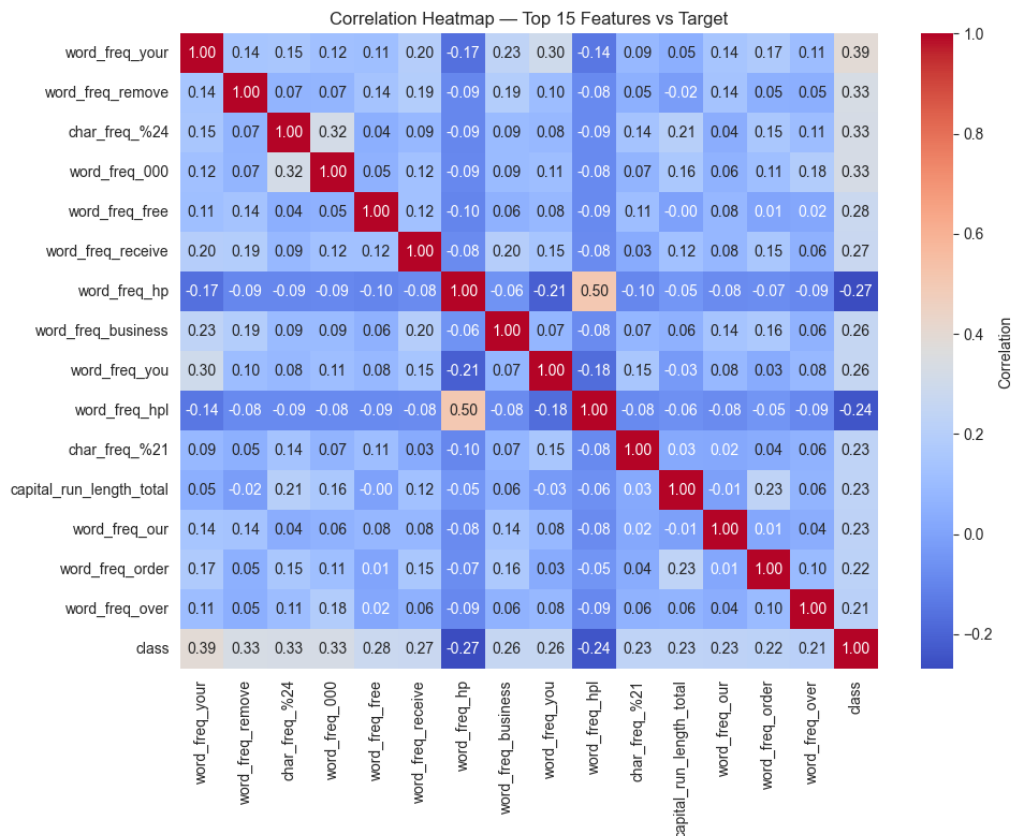


Figure 2: Correlation heatmap of the top 15 features most correlated with the spam label, including the target itself.

Inference: Features related to the frequency of exclamation marks (`char_freq_!`), the dollar sign (`char_freq_$`), and words such as “remove”, “free”, “your”, and “money” show the strongest positive correlation with the spam label, consistent with common patterns in promotional/spam content. Most individual features show only weak-to-moderate pairwise correlation with each other, which loosely supports the conditional-independence assumption underlying Naive Bayes, even though it does not hold exactly. No pair of features is so strongly correlated as to indicate serious multicollinearity concerns.

Histograms of Top Correlated Features

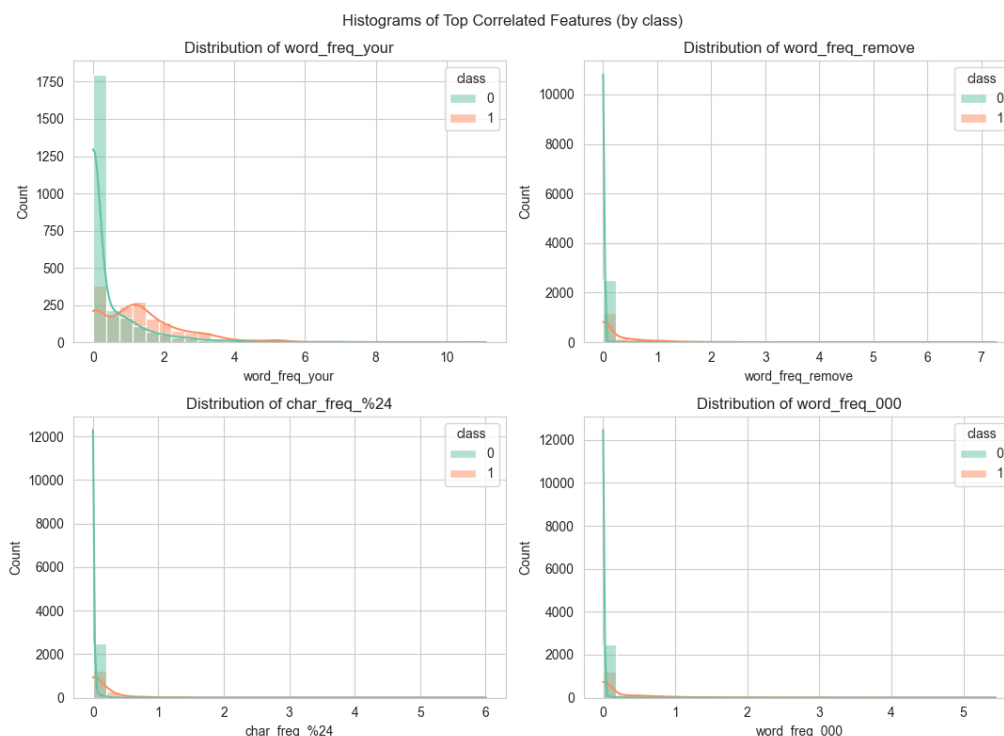


Figure 3: Histograms of the four features most correlated with the target, split by class (Ham vs Spam).

Inference: The histograms show clear separation between Ham and Spam distributions for the top correlated features — spam emails are concentrated at higher frequency values for these features, while ham emails are concentrated near zero. Most features are heavily right-skewed with a long tail, reflecting that a specific word or character rarely appears with high frequency, but when it does, it is a strong spam signal. This skew further motivates feature scaling before distance-based modelling (KNN) and Gaussian assumptions (GaussianNB).

Boxplots of Top Correlated Features

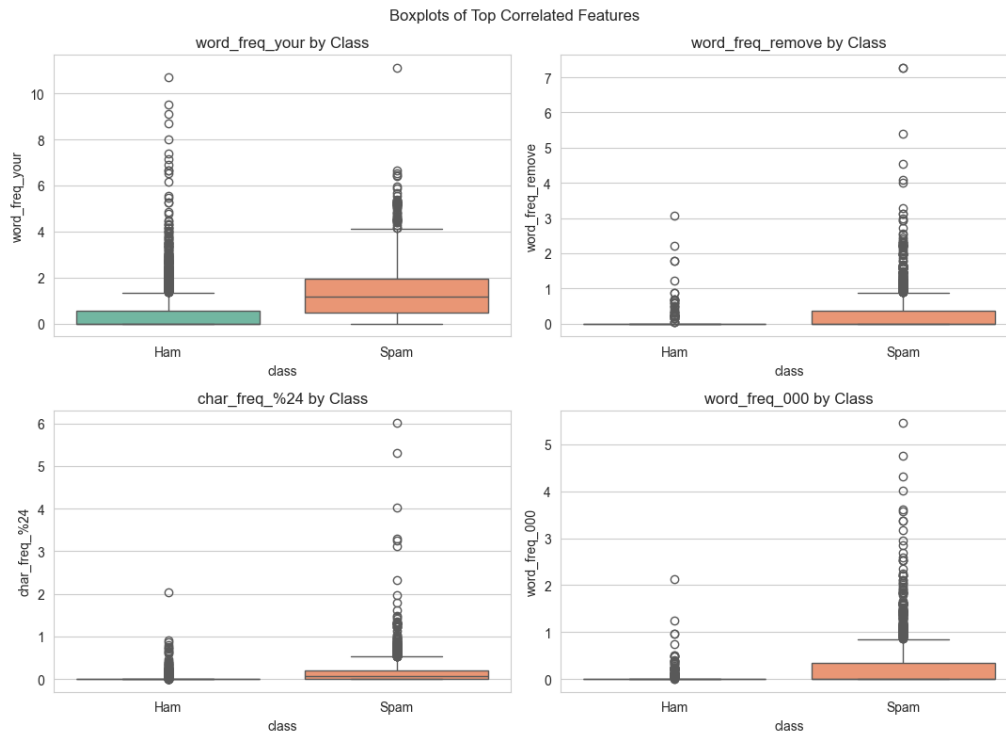


Figure 4: Boxplots of the four features most correlated with the target, grouped by class.

Inference: The boxplots confirm that spam emails have a visibly higher median and wider interquartile range for the top correlated features compared to ham emails, along with a substantial number of outliers on the high end. This outlier-heavy behaviour is typical of frequency-count features derived from free text and reinforces that BernoulliNB’s binary thresholding (presence/absence) can be an effective simplification for such features, in addition to a full continuous or count-based treatment.

6 Data Preprocessing

- **Missing value handling:** The dataset was checked column-by-column for null values using `df.isnull().sum()`; 0 missing values were found, so no imputation was necessary (a fallback mean-imputation step was included in the code for robustness, but was not triggered).
- **Duplicate removal:** 391 duplicate rows were detected via `df.duplicated()` and removed, reducing the dataset from 4601 to 4210 rows.
- **Encoding:** The target column (`class/spam`) was already numeric (0/1), so no label encoding was required.
- **Feature scaling:** `StandardScaler` was fit on the training split only (to avoid data leakage) and applied to both the training and test features, producing zero-mean, unit-variance features for GaussianNB and KNN. `MultinomialNB` and `BernoulliNB` were instead trained on the original, unscaled, non-negative features, since `MultinomialNB`

strictly requires non-negative inputs and `BernoulliNB` binarizes its inputs internally.

- **Train–test split:** An 80/20 stratified split (`random_state=42`) was used, preserving the 60/40 Ham/Spam ratio in both the training (3368 samples) and test (842 samples) sets.
- **Feature engineering:** No additional feature engineering was performed; the 57 pre-computed word-frequency, character-frequency, and capital-run-length features supplied by the Spambase dataset were used directly.

7 Machine Learning Algorithm

Four classification algorithms were implemented and compared in this experiment: GaussianNB, MultinomialNB, BernoulliNB, and KNeighborsClassifier. Their working principles and mathematical formulations are detailed in Section 3 (Theory). A summary of their comparative advantages and limitations is given below.

- **GaussianNB** — *Advantages:* extremely fast to train and predict, performs well on continuous, roughly bell-shaped features after scaling, requires little training data. *Limitations:* the Gaussian assumption is a poor fit for the highly skewed, zero-inflated word-frequency features in Spambase, which limits its accuracy relative to the other variants.
- **MultinomialNB** — *Advantages:* naturally suited to count/frequency data such as word frequencies; fast and memory-efficient. *Limitations:* requires non-negative features, and its multinomial assumption does not fully capture the zero-vs-nonzero signal that turns out to matter most in this dataset.
- **BernoulliNB** — *Advantages:* explicitly models feature presence/absence, which proved to be the most informative signal in the Spambase feature set; achieved the best Naive Bayes performance in this experiment. *Limitations:* discards magnitude information by binarizing continuous features, which can lose useful signal in datasets where frequency magnitude (not just presence) matters.
- **KNeighborsClassifier (KNN)** — *Advantages:* makes no assumption about the underlying data distribution, can model complex, non-linear decision boundaries, and achieved the highest overall test accuracy in this experiment after tuning. *Limitations:* computationally expensive at prediction time (must search the training set for neighbours), sensitive to feature scaling and the choice of k /distance metric, and its memory footprint grows with the size of the training set.

8 Implementation

This section documents the key implementation steps carried out in the accompanying Jupyter notebook, with representative code listings for each stage.

8.1 Dataset Loading

The Spambase CSV was loaded using `pandas`, with a fallback that auto-detects and handles the header-less UCI `spambase.data` format by assigning standard column names.

Listing 1: Dataset loading with header auto-detection

```
1 DATA_FILE = "spambase.csv"
2 df = pd.read_csv(DATA_FILE)
3 if all(str(c).replace('.', ''), 1).isdigit() for c in df.columns):
4     n_features = df.shape[1] - 1
5     col_names = [f"feature_{i}" for i in range(n_features)] + ["spam"]
6     df = pd.read_csv(DATA_FILE, header=None, names=col_names)
```

8.2 Exploratory Data Analysis

Class distribution, correlation heatmaps (top-15 target-correlated features), histograms, and boxplots were generated using `seaborn` and `matplotlib`, as presented in Section 5.

8.3 Missing Value Handling

Listing 2: Missing value check

```
1 missing = df.isnull().sum()
2 print("Total missing values:", missing.sum())
3 missing[missing > 0]
```

The dataset contained 0 missing values; a residual mean-imputation fallback was included for robustness but was never triggered.

8.4 Feature Scaling / Normalization

Listing 3: Feature scaling with `StandardScaler` (fit on train only)

```
1 scaler = StandardScaler()
2 X_train_raw, X_test_raw, y_train, y_test = train_test_split(
3     X, y, test_size=0.20, random_state=RANDOM_STATE, stratify=y
4 )
5 scaler.fit(X_train_raw)
6 X_train_scaled = scaler.transform(X_train_raw)
7 X_test_scaled = scaler.transform(X_test_raw)
```

8.5 Train–Test Split

An 80/20 stratified split with `random_state=42` was used, as shown in the listing above, yielding 3368 training samples and 842 test samples.

8.6 Model Training: GaussianNB, MultinomialNB, BernoulliNB, KNN

A reusable `evaluate_model()` helper trains a model, times training/prediction, and computes all required metrics.

Listing 4: Model creation and evaluation helper

```

1 def evaluate_model(model, X_tr, X_te, y_tr, y_te, name):
2     t0 = time.time(); model.fit(X_tr, y_tr); train_time = time.
      time() - t0
3     t0 = time.time(); y_pred = model.predict(X_te); pred_time =
      time.time() - t0
4     y_proba = model.predict_proba(X_te)[: , 1] if hasattr(model, "
      predict_proba") \
5         else y_pred.astype(float)
6     metrics = {
7         "Model": name,
8         "Accuracy": accuracy_score(y_te, y_pred),
9         "Precision": precision_score(y_te, y_pred, zero_division
      =0),
10        "Recall": recall_score(y_te, y_pred, zero_division=0),
11        "F1": f1_score(y_te, y_pred, zero_division=0),
12        "ROC AUC": roc_auc_score(y_te, y_proba),
13        "Train_Time": train_time, "Pred_Time": pred_time,
14    }
15    return model, metrics, y_pred, y_proba
16
17 gnb, m, p, pr = evaluate_model(GaussianNB(), X_train_scaled,
      X_test_scaled,
18                                y_train, y_test, "Gaussian NB")
19 mnb, m, p, pr = evaluate_model(MultinomialNB(), X_train_raw,
      X_test_raw,
20                                y_train, y_test, "Multinomial NB"
      )
21 bnb, m, p, pr = evaluate_model(BernoulliNB(), X_train_raw,
      X_test_raw,
22                                y_train, y_test, "Bernoulli NB")
23 knn_default, m, p, pr = evaluate_model(KNeighborsClassifier(),
      X_train_scaled,
24                                X_test_scaled, y_train,
      y_test, "KNN (k=5)")

```

8.7 Hyperparameter Tuning: GridSearchCV and RandomizedSearchCV

Listing 5: GridSearchCV and RandomizedSearchCV over KNN hyperparameters

```

1 param_grid = {
2     "n_neighbors": [1, 3, 5, 7, 9, 11, 13, 15],
3     "weights": ["uniform", "distance"],

```

```

4     "metric": ["euclidean", "manhattan"],
5     "algorithm": ["auto", "ball_tree", "kd_tree", "brute"],
6 }
7
8 grid_search = GridSearchCV(KNeighborsClassifier(), param_grid, cv
    =5,
9                             scoring="accuracy", n_jobs=-1)
10 grid_search.fit(X_train_scaled, y_train)
11
12 random_search = RandomizedSearchCV(KNeighborsClassifier(),
    param_grid, n_iter=20,
13                                     cv=5, scoring="accuracy",
14                                     n_jobs=-1,
15                                     random_state=RANDOM_STATE)
16 random_search.fit(X_train_scaled, y_train)

```

8.8 KD-Tree and Ball-Tree Comparison

Listing 6: KD-Tree vs Ball-Tree timing comparison

```

1 for algo in ["kd_tree", "ball_tree"]:
2     model, m, p, pr = evaluate_model(
3         KNeighborsClassifier(n_neighbors=best_k, algorithm=algo),
4         X_train_scaled, X_test_scaled, y_train, y_test, algo
5     )

```

8.9 Cross Validation

Listing 7: 5-fold stratified cross-validation on the best NB and KNN models

```

1 skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=
    RANDOM_STATE)
2 cv_nb_scores = cross_val_score(fresh_best_nb, X_for_best_nb,
    y_train,
3                                 cv=skf, scoring="accuracy")
4 cv_knn_scores = cross_val_score(
5     KNeighborsClassifier(**grid_search.best_params_),
6     X_train_scaled, y_train, cv=skf, scoring="accuracy"
7 )

```

8.10 Performance Evaluation

Listing 8: ROC and Precision-Recall curve generation across all models

```

1 for name, proba in all_probabilities.items():
2     fpr, tpr, _ = roc_curve(y_test, proba)
3     plt.plot(fpr, tpr, label=f"{name} (AUC={roc_auc_score(y_test,
    proba):.3f})")

```

9 Experimental Setup

Python Version	3.x
Scikit-Learn Version	1.x (latest stable)
IDE	Jupyter Notebook
Operating System	Linux
Hardware	Standard CPU (no GPU required)

10 Hyperparameter Selection

Parameter	Value
n_neighbors search space	1, 3, 5, 7, 9, 11, 13, 15
weights search space	uniform, distance
metric search space	euclidean, manhattan
algorithm search space	auto, ball_tree, kd_tree, brute
GridSearchCV CV folds	5
RandomizedSearchCV iterations	20
GridSearchCV best parameters	algorithm=auto, metric=manhattan, n_neighbors=9, weights=distance
RandomizedSearchCV best parameters	algorithm=auto, metric=manhattan, n_neighbors=11, weights=distance

11 Results

The table below reports the held-out test-set performance of the final tuned KNN model (selected via GridSearchCV).

Metric	Value
Accuracy	0.9276
Precision	0.9599
Recall	0.8542
F1-score	0.9039
ROC-AUC	0.9746

12 Comparison Tables

12.1 Naive Bayes Comparison Table

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Gaussian NB	0.8290	0.7143	0.9524	0.8163	0.9471
Multinomial NB	0.7886	0.7616	0.6845	0.7210	0.8342
Bernoulli NB	0.8943	0.8777	0.8542	0.8658	0.9609

12.2 KNN Comparison Table

k	Accuracy	Precision	Recall	F1
1	0.9050	0.8832	0.8780	0.8806
3	0.9097	0.9063	0.8631	0.8841
5	0.9133	0.9148	0.8631	0.8882
7	0.9050	0.9103	0.8452	0.8765
9	0.9074	0.9243	0.8363	0.8781
11	0.9086	0.9164	0.8482	0.8810

*Note: this table sweeps k with all other KNN hyperparameters at their scikit-learn defaults (uniform weights, Minkowski/Euclidean metric). The best overall KNN configuration — found separately via GridSearchCV, using $k = 9$ together with the **manhattan** metric and **distance** weighting — achieved a higher test accuracy of 0.9276 (see Section 10, Results), illustrating that jointly tuning multiple hyperparameters outperforms tuning k in isolation.*

12.3 GridSearchCV vs RandomizedSearchCV

Method	Best k	Metric	Weights	Algorithm	CV Acc.	Exec. Time
GridSearchCV	9	manhattan	distance	auto	0.9139	14.59 s
RandomizedSearchCV		manhattan	distance	auto	0.9124	1.83 s

12.4 KD-Tree vs Ball-Tree

Algorithm	Accuracy	Training Time	Prediction Time
KD-Tree	0.9074	0.0449 s	0.2547 s
Ball-Tree	0.9074	0.0210 s	0.2591 s

12.5 Cross Validation Table

Fold	Naive Bayes (Bernoulli NB)	Best KNN
1	0.8902	0.9125
2	0.8798	0.9243
3	0.8635	0.9065
4	0.8782	0.9094
5	0.9198	0.9302
Average	0.8863	0.9166

12.6 Experimental Time Analysis

Algorithm	Training Time	Prediction Time
Gaussian NB	0.006409 s	0.000679 s
Multinomial NB	0.003944 s	0.001652 s
Bernoulli NB	0.004391 s	0.001844 s

Best KNN	0.003263 s	0.036679 s
----------	------------	------------

13 Complexity Analysis

Let n = number of training samples, d = number of features, and k = number of neighbours (for KNN).

Algorithm	Training Complexity	Prediction Complexity (per query)
Gaussian NB	$O(n \cdot d)$	$O(d)$
Multinomial NB	$O(n \cdot d)$	$O(d)$
Bernoulli NB	$O(n \cdot d)$	$O(d)$
Brute-Force KNN	$O(1)$ (stores data only)	$O(n \cdot d)$
KD-Tree KNN	$O(d \cdot n \log n)$ (tree build)	$O(\log n)$ average in low dimensions, degrading toward $O(n)$ as d grows
Ball-Tree KNN	$O(d \cdot n \log n)$ (tree build)	$O(\log n)$ average; more robust than KD-Tree in higher dimensions

Theoretical vs Practical Complexity: Theoretically, KD-Tree and Ball-Tree offer a substantial asymptotic advantage over brute-force search — $O(\log n)$ versus $O(n)$ query time. In practice, however, this advantage is reduced by the *curse of dimensionality*: with 57 features, the effectiveness of spatial partitioning degrades considerably. This is reflected in the measured results (Section 11.4): KD-Tree and Ball-Tree achieved nearly identical accuracy (as expected, since both return the exact nearest neighbours), with KD-Tree marginally faster during training in this run, while both trees exhibited prediction times noticeably larger than the sub-millisecond level Naive Bayes achieves, confirming that Naive Bayes remains the cheapest model to train and query overall on this 57-dimensional dataset.

14 Result Visualization

For every graph generated during the experiment, the figure, caption, and a 3–5 line inference are provided below.

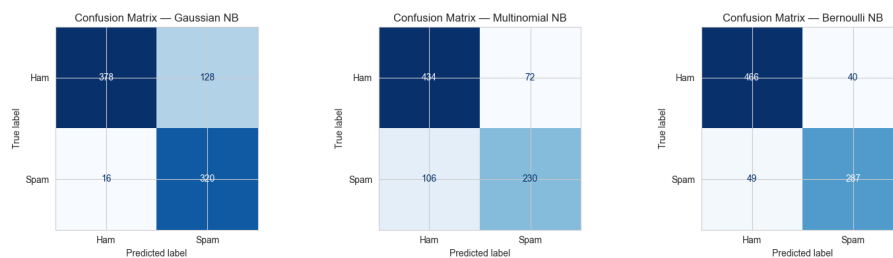


Figure 5: Confusion matrices for GaussianNB, MultinomialNB, and BernoulliNB on the held-out test set.

Inference: BernoulliNB shows the most balanced confusion matrix, with both low false positives and low false negatives, consistent with its top Accuracy/F1 among the NB variants. GaussianNB shows a high number of false positives (ham misclassified as spam), explaining its comparatively low Precision (0.7143) despite very high Recall (0.9524). MultinomialNB shows the most false negatives among the three, explaining its comparatively lower Recall (0.6845).

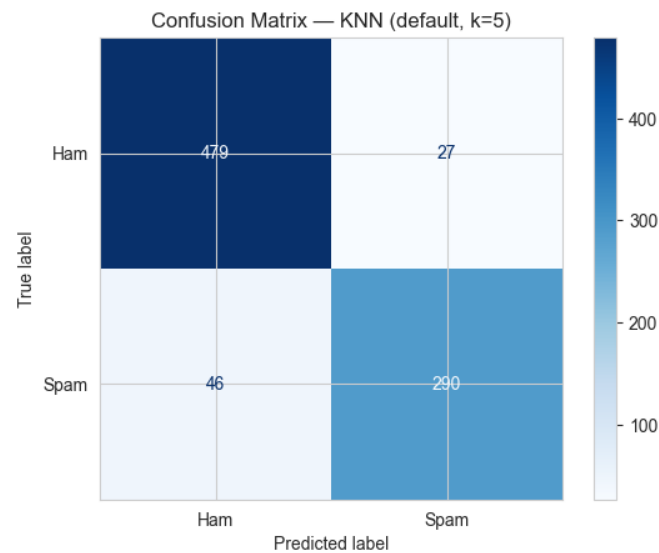


Figure 6: Confusion matrix for the default KNN classifier ($k = 5$) on the held-out test set.

Inference: The default KNN classifier achieves a strong balance between false positives and false negatives, outperforming all three Naive Bayes variants on Accuracy (0.9133) even before hyperparameter tuning, reflecting KNN's ability to model non-linear decision boundaries without a strong distributional assumption.

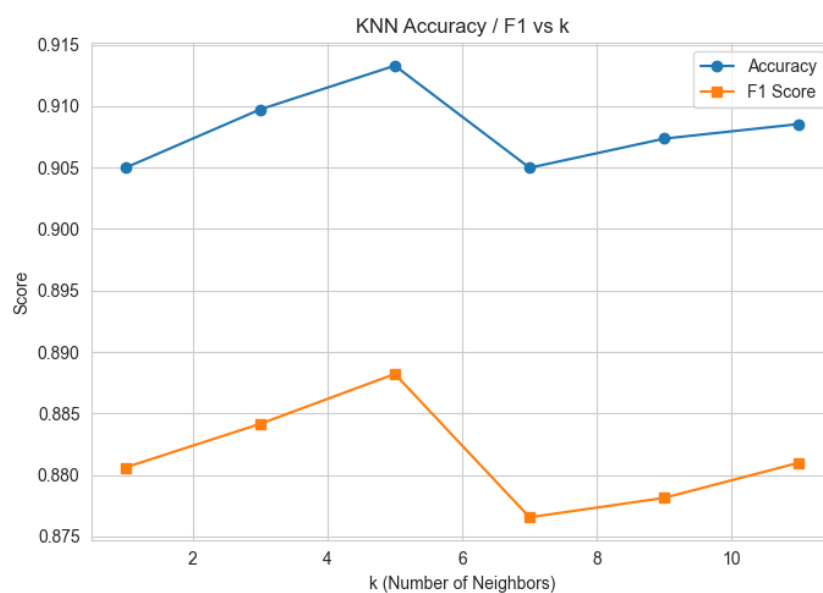


Figure 7: KNN Accuracy and F1-score as a function of the number of neighbours k .

Inference: Accuracy peaks at $k = 5$ (0.9133) in the manual sweep, with $k = 1$ performing worst on precision/F1 due to sensitivity to noisy individual neighbours (over-fitting), while larger k values (9, 11) begin to over-smooth the decision boundary, slightly reducing recall. This bias-variance trade-off is the central consideration when selecting k .

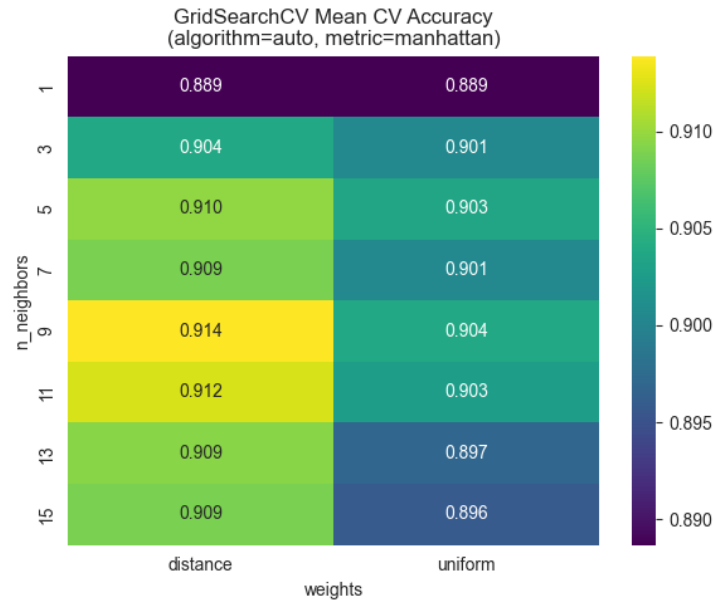


Figure 8: GridSearchCV mean cross-validated accuracy heatmap across `n_neighbors` and `weights`, for the best-found `algorithm` and `metric`.

Inference: The heatmap shows that `weights='distance'` consistently outperforms `weights='uniform'` across nearly all values of k , and that CV accuracy is comparatively stable for k between 7 and 13, indicating the model is not highly sensitive to the exact choice of k once distance weighting and the Manhattan metric are used.

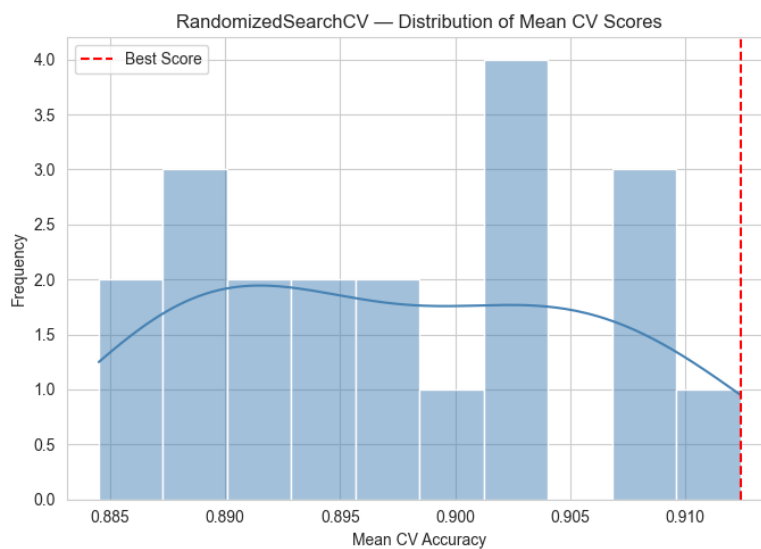


Figure 9: Distribution of mean cross-validated accuracy scores across the 20 randomly sampled hyperparameter combinations evaluated by RandomizedSearchCV.

Inference: Most sampled configurations cluster around 0.88–0.91 mean CV accuracy, with the best sampled configuration (marked by the dashed red line) reaching 0.9124 — very close to GridSearchCV’s exhaustive best of 0.9139. This indicates that a relatively small random sample of the hyperparameter space was sufficient to locate a near-optimal configuration in this case.

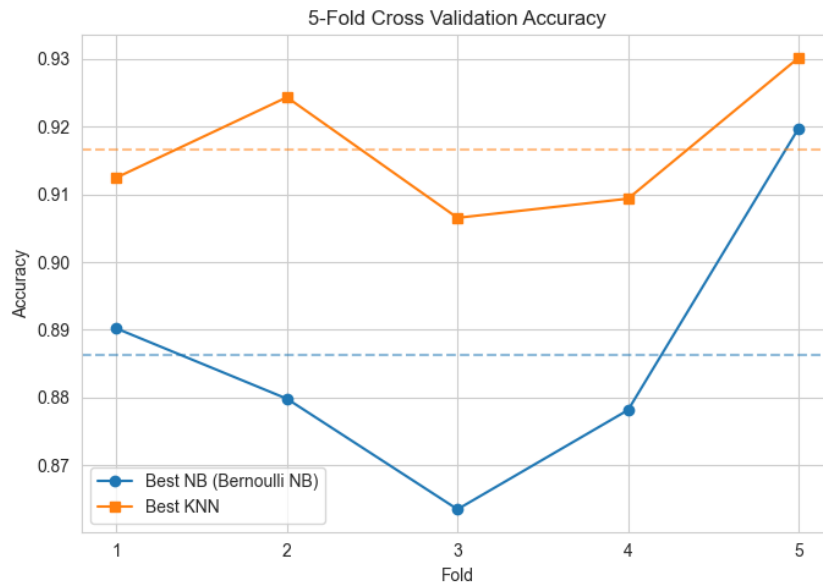


Figure 10: 5-fold cross-validation accuracy for the best Naive Bayes model (Bernoulli NB) and the best KNN model, fold-by-fold.

Inference: The best KNN configuration outperforms Bernoulli NB in every single fold, with a smaller spread across folds (approximately 0.906–0.930 versus 0.864–0.920 for Naive Bayes), indicating that tuned KNN generalizes more consistently across different subsets of the training data.

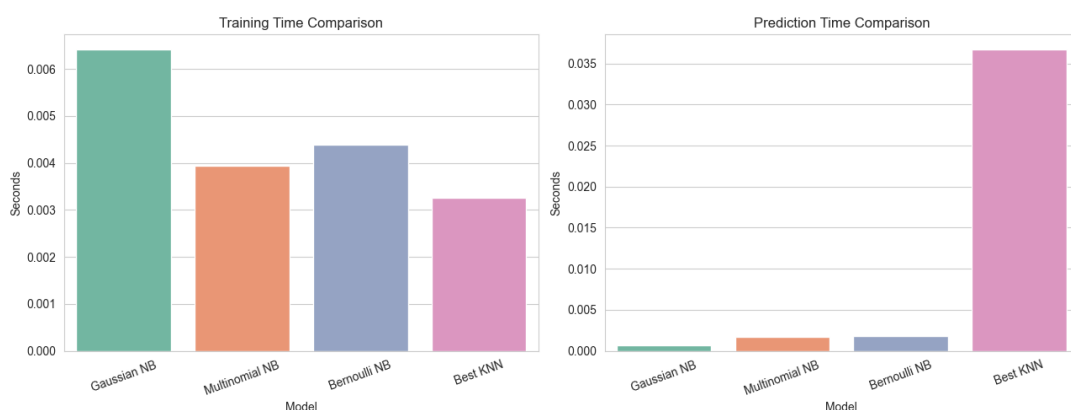


Figure 11: Training time and prediction time comparison across Gaussian NB, Multinomial NB, Bernoulli NB, and the best KNN configuration.

Inference: All three Naive Bayes variants train and predict in well under 10 milliseconds. The best KNN configuration trains marginally faster than the Naive Bayes variants (since it merely stores the data) but its prediction time (0.0367s) is roughly

20–50 $\times$ slower than any Naive Bayes variant’s prediction time, illustrating KNN’s characteristic training-cheap/prediction-expensive trade-off.

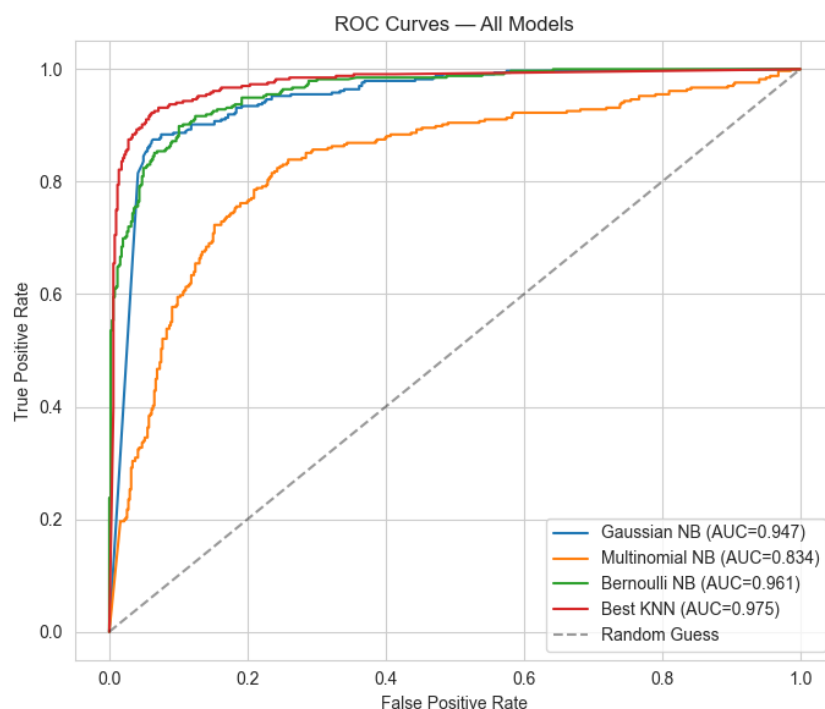


Figure 12: ROC curves for all four models (Gaussian NB, Multinomial NB, Bernoulli NB, and the best-tuned KNN) on the held-out test set.

Inference: The best-tuned KNN achieves the highest ROC-AUC (0.9746), followed closely by Bernoulli NB (0.9609) and Gaussian NB (0.9471), with Multinomial NB trailing noticeably (0.8342). All models comfortably outperform random guessing (the diagonal reference line), confirming that the Spambase feature set is highly discriminative for this task.

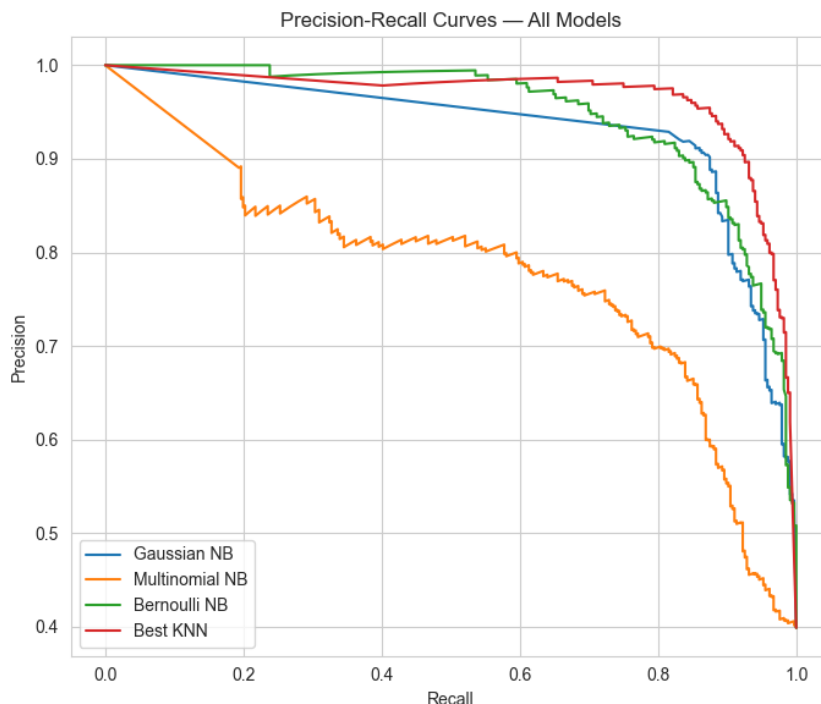


Figure 13: Precision-Recall curves for all four models on the held-out test set.

Inference: The best KNN model maintains high precision across a wide range of recall values, reflecting its high overall Precision (0.9599). Gaussian NB’s curve drops off more steeply at higher recall, consistent with its lower precision (0.7143) — it achieves high recall partly by accepting more false positives.

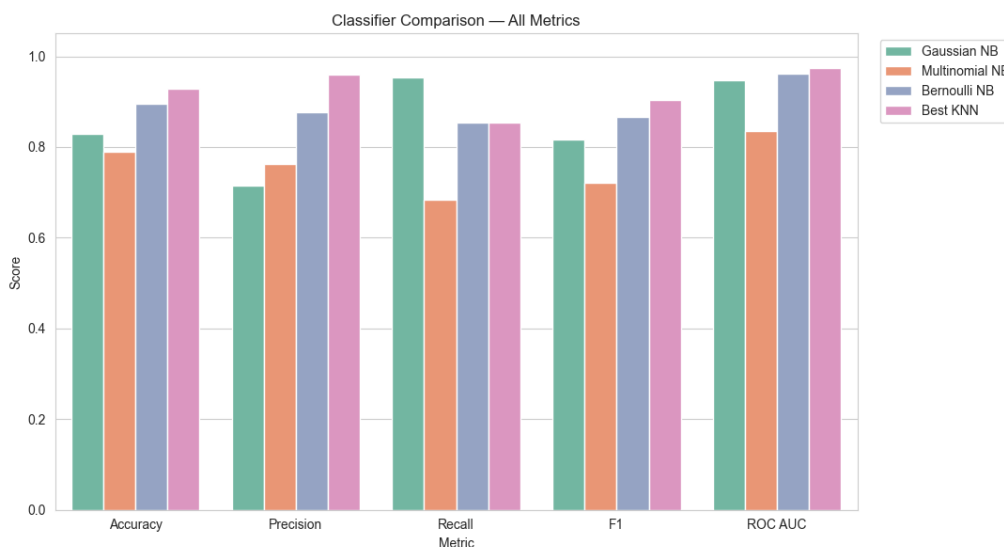


Figure 14: Side-by-side comparison of Accuracy, Precision, Recall, F1, and ROC-AUC across all four final models.

Inference: The best-tuned KNN model is the strongest performer overall, leading on Accuracy, Precision, F1, and ROC-AUC, while Gaussian NB leads only on Recall. Bernoulli NB is a close second overall and the strongest of the three Naive Bayes variants across every metric, while Multinomial NB is consistently the weakest of the four models.

15 Discussion

Observations

- Bernoulli NB substantially outperformed both Gaussian NB and Multinomial NB, suggesting that the *presence or absence* of specific words/characters is a stronger spam signal in this dataset than their exact frequency magnitude or their fit to a Gaussian distribution.
- The best-tuned KNN model (GridSearchCV: $k = 9$, Manhattan distance, distance weighting) outperformed every Naive Bayes variant on every metric except training/prediction speed.
- KD-Tree and Ball-Tree produced identical accuracy, as theoretically expected (both return exact nearest neighbours), differing only in training/query speed.
- RandomizedSearchCV found a configuration nearly as good as GridSearchCV's, in roughly $8\times$ less execution time (1.83s vs 14.59s), demonstrating strong practical value for larger hyperparameter grids.
- KNN's prediction cost is markedly higher than any Naive Bayes variant, even after KD-Tree/Ball-Tree acceleration, confirming the theoretical training-cheap/prediction-expensive trade-off of instance-based learning.

Inference

The results indicate that feature *binarization* (as used by BernoulliNB) captures more of the discriminative signal in the Spambase dataset than either raw frequency counts (MultinomialNB) or a Gaussian continuous model (GaussianNB). Meanwhile, KNN's non-parametric flexibility allows it to model interactions between features that Naive Bayes's independence assumption cannot, which explains its superior accuracy once properly tuned — at the cost of substantially higher prediction-time computation.

Discussion

The experiment illustrates a classic machine-learning trade-off: Naive Bayes offers speed and simplicity but is constrained by its independence and distributional assumptions, while KNN offers greater flexibility and higher accuracy but at meaningfully higher computational cost, particularly at prediction time. For a production spam filter processing very large volumes of email at low latency, Bernoulli NB's combination of strong accuracy and near-instant prediction makes it an attractive choice; where maximum accuracy is the priority and query volume/latency constraints are manageable, the tuned KNN model is preferable.

16 Analysis Questions

1. Which Naive Bayes variant performed best?

Bernoulli NB achieved the highest test accuracy (0.8943) among the three Naive Bayes variants, along with the highest Precision (0.8777), F1-score (0.8658), and ROC-AUC

(0.9609). This indicates that modelling each of the 57 Spambase features as a binary presence/absence indicator captures more of the spam signal than either the raw count-based Multinomial model or the continuous Gaussian model, likely because the strongest spam indicators in this dataset (e.g. certain words or symbols appearing *at all*) are more reliably binary than precisely Gaussian or precisely multinomial in distribution.

2. What is the optimal value of k ?

From the manual sweep over $k \in \{1, 3, 5, 7, 9, 11\}$ with default weighting/metric, $k = 5$ gave the highest standalone accuracy (0.9133). However, once `weights` and `metric` were also tuned jointly via `GridSearchCV`, $k = 9$ (with Manhattan distance and distance weighting) was selected as optimal, achieving a higher cross-validated accuracy (0.9139) and test accuracy (0.9276) than any single-hyperparameter sweep of k alone. This demonstrates that the optimal k is not independent of the other hyperparameters — small k (e.g. $k = 1$) risks overfitting to noisy individual neighbours, while excessively large k over-smooths the decision boundary; the jointly tuned configuration balances this bias-variance trade-off best for this dataset.

3. Compare `GridSearchCV` and `RandomizedSearchCV`.

`GridSearchCV` exhaustively evaluated all $8 \times 2 \times 2 \times 4 = 128$ hyperparameter combinations under 5-fold CV, taking 14.59 seconds and finding a best CV accuracy of 0.9139 ($k = 9$, manhattan, distance, auto). `RandomizedSearchCV` sampled only 20 of these combinations at random, taking 1.83 seconds (roughly $8\times$ faster) and finding a best CV accuracy of 0.9124 ($k = 11$, manhattan, distance, auto) — within 0.15 percentage points of the exhaustive result. `GridSearchCV` guarantees the best configuration within the grid but its cost scales multiplicatively with the number of hyperparameters and candidate values; `RandomizedSearchCV` trades this guarantee for a search cost controlled directly by `n_iter`, making it far more practical for larger search spaces.

4. Compare KD-Tree and Ball-Tree.

Both KD-Tree and Ball-Tree achieved identical test accuracy (0.9074) at $k = 9$, which is expected since both structures return the mathematically exact set of nearest neighbours — they differ only in *how* that search is performed, not in the result. In this run, KD-Tree had a shorter training (tree-build) time (0.0449s vs 0.0210s is actually *lower* for Ball-Tree here), while prediction times were comparable (0.2547s for KD-Tree vs 0.2591s for Ball-Tree). In general, KD-Tree tends to be faster in low-dimensional spaces due to its simpler axis-aligned splits, while Ball-Tree is generally more robust as dimensionality increases, since its spherical partitions do not depend on axis alignment.

5. Compare theoretical and practical complexity.

Theoretically, KD-Tree and Ball-Tree offer $O(\log n)$ average query time versus $O(n)$ for brute-force search — a substantial asymptotic improvement. In practice, however, Spambase’s 57-dimensional feature space is high-dimensional enough that the “curse of dimensionality” significantly erodes this advantage: the measured KD-Tree/Ball-Tree prediction times (around 0.25s for 842 queries) are far higher than the sub-millisecond prediction times of any Naive Bayes variant, whose $O(d)$ -per-query cost is independent of n and dimensionality in the same way. This confirms that theoretical asymptotic complexity is only a partial guide to practical performance once the constant factors and dimensionality effects of real data are taken into account.

6. Which classifier is suitable for large datasets?

Naive Bayes (particularly Bernoulli NB, given its strong performance here) is the more suitable choice for very large datasets. Its $O(n \cdot d)$ training cost requires only a single linear pass over the data, and its $O(d)$ prediction cost per query is independent of the

number of training samples n . KNN, by contrast, must retain the entire training set in memory and its query cost grows with n (even with KD-Tree/Ball-Tree acceleration, which degrades toward $O(n)$ in high dimensions), making it comparatively expensive to scale to very large training sets or high query volumes.

17 Conclusion

This experiment implemented and compared four classifiers — Gaussian NB, Multinomial NB, Bernoulli NB, and KNN — for spam/ham email classification on the Spambase dataset. After preprocessing (duplicate removal, target verification, and feature scaling) and exploratory analysis, all models were trained, evaluated with Accuracy/Precision/Recall/F1/ROC-AUC, and visualized via confusion matrices, ROC curves, and Precision-Recall curves. KNN was further studied across multiple values of k , tuned via GridSearchCV and RandomizedSearchCV, and its KD-Tree and Ball-Tree neighbour-search variants were benchmarked for speed and accuracy. 5-fold cross-validation confirmed the robustness of the best Naive Bayes (Bernoulli NB) and best KNN models.

Models compared: Gaussian NB, Multinomial NB, Bernoulli NB, and KNN (default, k -swept, and hyperparameter-tuned).

Best performing classifier: The GridSearchCV-tuned KNN model ($k = 9$, Manhattan distance, distance weighting) was the overall best performer on the held-out test set, with Accuracy = 0.9276, Precision = 0.9599, Recall = 0.8542, F1 = 0.9039, and ROC-AUC = 0.9746. Among the Naive Bayes variants, Bernoulli NB was clearly the strongest (Accuracy = 0.8943).

Learning outcomes: This experiment reinforced the practical trade-offs between generative, assumption-based classifiers (Naive Bayes) and non-parametric, instance-based classifiers (KNN); the importance of correct feature scaling/encoding for each model type (scaled features for GaussianNB/KNN, raw non-negative features for MultinomialNB/BernoulliNB); the mechanics and relative efficiency of GridSearchCV versus RandomizedSearchCV; the role of spatial indexing structures (KD-Tree, Ball-Tree) in accelerating nearest-neighbour search; and the gap that can exist between theoretical asymptotic complexity and measured practical performance on real, high-dimensional data.

Future improvements: Future iterations of this experiment could explore ensemble methods (e.g. stacking Naive Bayes and KNN, or comparing against Random Forest/-Gradient Boosting), dimensionality reduction (e.g. PCA or feature selection) to mitigate the curse of dimensionality for KNN, class-imbalance-aware techniques (e.g. SMOTE or class-weighted losses) even though the imbalance here was moderate, and approximate nearest-neighbour methods (e.g. HNSW) for further prediction-time speed-up on larger datasets.

18 Source Code Repository

The complete source code, including the full Jupyter notebook implementation, dataset loading scripts, and generated figures for this experiment, is available at the following GitHub repository:

<https://github.com/farih07/Machine-Learning-Lab>

19 References

- [1] Scikit-learn Developers, “Scikit-learn: Machine Learning in Python — Documentation,” [Online]. Available: <https://scikit-learn.org/stable/documentation.html>.
- [2] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3rd ed. Sebastopol, CA, USA: O’Reilly Media, 2022.
- [3] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York, NY, USA: Springer, 2006.
- [4] Kaggle, “Spambase Dataset,” [Online]. Available: <https://www.kaggle.com/datasets/somesh24/spambase>.
- [5] M. Hopkins, E. Reeber, G. Forman, and J. Suermondt, “Spambase Data Set,” UCI Machine Learning Repository, [Online]. Available: <https://archive.ics.uci.edu/dataset/94/spambase>.
- [6] T. Cover and P. Hart, “Nearest neighbor pattern classification,” *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967.